Untested by Construction

Test Coverage of Write-Capable Tools in Open-Source LLM Agents

Rithik Satarla
rithiksatarla@gmail.com

August 4, 2026

Abstract

Large language model agents increasingly hold credentials for systems that change state: they issue refunds, delete records, push branches, and send mail. The dominant way to test such an agent is to record an HTTP response and replay it. A replay fixture, however, has no state, and so it cannot express the failure mode specific to write operations—an operation applied twice, or applied when it should have been refused. An agent that retries a refund it already completed passes a fixture-based suite and double-refunds in production.

We ask how much of the open-source agent ecosystem tests its write paths at all. We screened 45 Python-primary agent repositories, mechanically extracted framework-idiomatic tool definitions, and retained the 158 definitions whose bodies perform a detectable state mutation across the 13 repositories that had any. Using a deliberately generous coverage criterion—a tool is covered if its identifier appears anywhere in the repository's test sources, with no assertion or execution required—we find that 20.3% of write-capable tool definitions (32 of 158) are never referenced by any test. Restricting to the 7 repositories whose entire test suite was read gives 20.7%, so the estimate does not depend on sampling. Because the criterion is an upper bound on real coverage, 20.3% is a lower bound on what goes untested.

We then describe AgentCage, a stateful alternative: a passive HTTP interceptor plus an in-memory API model in which writes mutate state and subsequent reads observe the mutation. We pre-register an evaluation of whether this surfaces defects that replay fixtures cannot. All code, data, and the detector revision history are public.

Keywords: LLM agents; tool use; software testing; test coverage; mocking; state machines; empirical software engineering.

1 Introduction

An LLM agent with tool access is, operationally, a program that issues requests against external systems. A growing fraction of those requests are writes. Agent frameworks ship tools that commit to git, delete cloud objects, run shell commands, execute SQL, send email, and move money.

The testing practice these agents inherited comes from the world of read-mostly API clients: record a response, replay it. This works for a tool that fetches a document. It fails for a tool that moves money, and it fails in a specific, structural way. Consider an agent that issues a refund, fails to parse the response, and retries:

```
refund(charge_id, 5000) # fixture replies 200 OK
refund(charge_id, 5000) # fixture replies 200 OK <- suite passes
                        # Stripe would reply 400 charge_already_refunded
```

The fixture answers identically however many times it is asked, because it holds no state. The defect lives entirely in the state transition, so no assertion over the fixture's responses can detect it. The test suite is not merely incomplete; it is incapable of expressing the property.

This paper makes two contributions.

A measurement (Section 3). We quantify how much of the open-source agent ecosystem tests its write-capable tools, using a mechanical, reproducible procedure over 45 repositories. The headline is 20.3% of 158 write-capable tool definitions never referenced by any test. We report the result together with the seven revisions our detectors underwent, two of which removed false positives that would have made the finding look worse.

An artefact and a pre-registered evaluation (Section 4). We describe AgentCage: passive HTTP capture plus a stateful API model. We state in advance what would count as evidence that it surfaces defects fixtures cannot, and what would falsify that claim. The evaluation has not been run; no results are reported for it.

Scope. This is a study of *test coverage*, not of agent safety in deployment. We do not claim the uncovered tools are buggy, nor that any agent has caused harm. We claim only that a fifth of the write-capable tools we could identify are not touched by their projects' tests, under the weakest definition of "touched" we could construct.

2 Related work

Agent benchmarks evaluate whether agents *accomplish* tasks. ReAct [1] established interleaved reasoning and acting; Toolformer [2], Gorilla [3], and ToolLLM [4] study tool invocation. SWE-bench [5], WebArena [6], AgentBench [7], and GAIA [8] measure task success in code, web, and general settings. These measure capability. We measure whether the tool implementations themselves are tested by the projects that ship them—a software-engineering property of the codebase, orthogonal to how well a model uses the tool.

Closest to our artefact is ToolEmu [9], which uses a language model to emulate tool execution in order to surface agent risks without real side effects, and AgentDojo [10], which evaluates agents against prompt-injection attacks in environments with stateful tools. Both establish that consequences of agent writes deserve dedicated infrastructure. AgentCage differs in mechanism and in aim: our model is deterministic code rather than an LM emulation, which trades breadth of coverage for exact, inspectable state transitions and reproducible failures; and our target is the developer's own test suite rather than a red-teaming evaluation.

We are not aware of prior work quantifying test coverage of write-capable tools across open-source agent repositories, which is the gap Section 3 addresses.

3 Part A: how much is tested?

3.1 Sampling frame

We assembled 45 candidate repositories: recognisable Python agent projects spanning frameworks, autonomous agents, coding agents, and browser agents. The frame is a convenience sample, not a random sample of GitHub, and is restricted to Python-primary repositories because every detector is a Python source pattern that would silently under-detect on other languages.

This biases the result toward well-maintained projects, which if anything biases measured coverage *upward*: the long tail of small agent repositories is unlikely to be better tested than CrewAI

or LlamaIndex.

3.2 Screening: what enters the denominator

A repository enters the analysed set only if the detectors find at least one *write-capable tool definition*. Screening is mechanical: 32 of 45 candidates were excluded by code, leaving 13. Screening out is not a claim that a project performs no writes; it means the project exposes no framework-idiomatic tool definition whose body performs a detectable mutation. Section 3.7 treats this as the study's principal limitation.

3.3 Detectors

Tool definitions are recognised in three styles: **decorator** (`@tool`, `@agent.tool`, `@mcp.tool()`, `@registry.action`, `@function_tool`, `@kernel_function`), **class** (a base named `*BaseTool`, `*Toolkit`, `*ToolSpec`, `*BaseAction`), and **schema** (a JSON tool schema pairing a `name` with `parameters` or `input_schema`). CLI decorators such as `@click.command` are deliberately excluded, since matching them would classify ordinary command-line programs as agents.

A definition is write-capable if its body matches at least one non-ambiguous mutation marker across ten categories: destructive HTTP verbs, filesystem mutation, file opens in write mode, shell execution, SQL and ORM writes, cloud SDK writes, version-control writes, outbound messaging, and payment operations. A bare HTTP `POST` is placed in a separate tier and *excluded*, because many read-only search and inference endpoints are queried with `POST`.

Two rules prevent a definition from serving as its own evidence. First, markers are matched only against the body *after* the signature: without this, `def delete_file(path)` satisfies a pattern intended to catch *calls* to `delete_file(`, and any tool whose name reads like a mutation is counted as one on the strength of its own name. Second, every occurrence of the tool's own identifier is neutralised before matching. Both rules were added after manual inspection revealed false positives that these exact mechanisms had produced.

3.4 Coverage criterion

For each analysed repository we read its test sources and count a write tool as **covered** if its identifier appears as a whole word anywhere in that corpus.

This bar is intentionally generous. It requires no assertion, no exercise of the destructive path, and no execution: a single import or an incidental mention in an unrelated test satisfies it. Coverage measured this way is therefore an *upper bound* on real coverage, and the uncovered fraction a *lower bound*. A tool this study calls untested is untested under the weakest available definition.

Unit of analysis. We count *tools*, not repositories. A repository-level criterion—"has at least one test touching at least one write tool"—is nearly vacuous: a project shipping 34 write tools with one of them referenced satisfies it. Counting tools places 158 items in the denominator rather than 13.

3.5 Results

Table 1 reports per-repository counts for the run of 2026-08-04. Across 13 repositories, 32 of 158 write-capable tool definitions (20.3%) are never referenced by any test.

Sensitivity. Restricting to the 7 repositories whose entire test suite was read—where a missed reference cannot be an artefact of sampling—gives 18 of 87 tools uncovered, or 20.7%. The agreement with the headline indicates the estimate is not driven by the read cap.

Repository	Tools	Covered	Uncovered	Uncov.	Test scan
agno-agi/agno	34	23	11	32%	capped
crewAIInc/crewAI	32	19	13	41%	full
camel-ai/camel	30	29	1	3%	full
run-llama/llama_index	13	13	0	0%	capped
geekan/MetaGPT	13	13	0	0%	full
langchain-ai/langchain-community	12	11	1	8%	capped
TransformerOptimus/SuperAGI	9	6	3	33%	full
Upsonic/Upsonic	6	4	2	33%	capped
Significant-Gravitas/AutoGPT	3	3	0	0%	capped
griptape-ai/griptape	3	3	0	0%	capped
microsoft/autogen	1	1	0	0%	full
openai/openai-agents-python	1	1	0	0%	full
aiwaves-cn/agents	1	0	1	100%	full
Total	**158**	**126**	**32**	**20.3%**	

Table 1. Write-capable tool definitions and their test coverage, for the run of 2026-08-04. "Test scan" is *full* where the repository's entire test suite was read and *capped* where the per-repository read limit bound. Generated directly from `part_a/results.json`.

Repository-level view. 1 of 13 repositories (7.7%) have no test touching any write tool: aiwaves-cn/agents, which ships no test files at all. We report this figure for completeness while noting it rests on a single repository and is far less informative than the tool-level result.

3.6 Detector revision history

The detectors were developed against the corpus, and each revision moved the headline. We record them because the final number is not interpretable without them.

Status of this analysis. Part A is **exploratory**, not pre-registered. The detectors were revised in response to inspecting their output, as Table 2 records. It should be read as a measurement with a documented derivation rather than a confirmatory test of a hypothesis fixed in advance. The Part B evaluation in Section 4 *is* pre-registered and has not been run.

3.7 Threats to validity

Construct validity is the dominant limitation. The study measures *framework-idiomatic tool definitions*. Agents with bespoke architectures—`aider`, `OpenHands`, `SWE-agent`, `gpt-engineer`—write to disk and execute shell commands continuously but expose no `@tool` or `BaseTool` surface, and therefore screen out. With 32 of 45 candidates excluded, the denominator of 13 reflects detector coverage as much as it reflects the ecosystem. The correct reading of our headline is "among tools we can mechanically identify," not "among all agent write paths."

Coverage inference. Name reference in a test file is not execution. A referenced tool may carry no assertion on its destructive path (inflating measured coverage); an unreferenced tool may be

Problem identified	Change and effect
Path filter matched substrings, so `openai.py` matched an "op" alternative	Match whole delimiter-separated path words; candidate files became genuine tool surfaces
File cap applied lexicographically, representing large repositories by whichever directory sorted first	Rank by tool-likeness before applying the cap
Reading only files whose *path* contained a tool keyword missed tools defined in files such as `controller/service.py`	Make every non-test source file eligible; 13→17 repositories analysed
`@registry.action` and MCP-style decorators absent from the vocabulary	Added; included in the above
`db_write` matched bare `.save(`, catching a PIL image save	Removed bare `.save(`; removed a false positive from the untested set
Patterns matched the definition line itself, classifying a tool as a write on the strength of `def delete_file(`	Strip signatures, neutralise self-reference; 17→13 repositories
Repository-level binary coverage too weak, and after the above rested on one repository	Switch unit of analysis to the tool; headline became 20.3% of 158

Table 2. Detector revisions. The final two rows removed findings that would have *raised* the reported figure.

exercised by an integration harness that never names it (deflating it). We argue the first error dominates because the criterion is so weak, which makes 20.3% a floor.

Browser and GUI writes. Agents that mutate state by clicking and submitting forms are not detected, since no marker recognises a DOM interaction.

Frameworks versus applications. Several screened-out projects ship the *mechanism* for defining tools rather than write tools themselves. Excluding them is correct, but it skews the analysed set toward tool catalogues.

Temporal validity. Files are read at `HEAD`. The measurement is of a moving target; results carry a timestamp and continuous integration re-runs the study weekly.

4 Part B: AgentCage

4.1 Design

Interceptor. Attaches through each HTTP library's own documented hook points (`httpx` event hooks, `requests` response hooks) and records method, URL, path, query, headers, body, and response status. It is strictly passive: it never rewrites a request, injects a response, or short-circuits the transport, and a test asserts that the caller receives exactly what the transport returned. Credential headers are redacted at capture time so traces are committable artefacts.

State model. A deterministic in-memory model of the money-moving subset of the Stripe API. Refunds increase `charge.amount_refunded` and set `charge.refunded` at parity with `amount_captured`; a subsequent `GET` observes the change. Refunds beyond the unrefunded amount are rejected with the real error code and message; a fully refunded charge rejects further

refunds; idempotency keys replay the original response and reject reuse with altered parameters. Identifiers are counter-derived and the clock is injectable, so runs are reproducible.

4.2 Fidelity boundary

Not modelled: payment-method validation, 3-D Secure, disputes, payouts, balance transactions, webhooks, connected accounts, field expansion, pagination cursors, rate limits, and Stripe's real identifier format. The model is sufficient to exercise double-refund, over-refund, refund-after-refund, and read-after-write, and no more. Stating this boundary matters: a mock's silent divergence from the real API is itself a source of false confidence.

4.3 Worked example

The smallest complete demonstration, produced by `part_b/demo_double_refund.py` and committed as `traces/example_double_refund.json`:

```
POST /v1/charges -> 200 ch_00000001 created, 5000 usd
POST /v1/refunds -> 200 amount_refunded 0 -> 5000
POST /v1/refunds -> 400 charge_already_refunded <- the retry
GET /v1/charges/ch_... -> 200 amount_refunded == 5000
```

Both `POST /v1/refunds` requests are byte-identical. A replay fixture answers the second with 200 and the customer is refunded twice; the stateful model refuses it. The captured trace carries `authorization: <redacted>`, which is what makes traces publishable artefacts.

4.4 Pre-registered evaluation

Status: not run. No results exist for this section. It is stated before the experiment precisely so that it cannot change afterwards.

H1. Agents that pass their existing replay-based suites exhibit write-path defects when the same paths are exercised against a stateful model.

Sample and exclusions. Drawn from the Part A analysed set, restricted to repositories whose tools perform payment-like writes. Pre-specified exclusions: no runnable suite; write tools requiring credentials we cannot substitute; tools that mutate only local filesystem state, giving the interceptor no HTTP surface to observe. Any agent added or dropped after its result is known will be reported as such.

Procedure. For each agent: run its own suite unmodified (expected: passes; an agent whose suite already fails is excluded and reported); re-run its write paths through the interceptor against the stateful model; grade every request by the resolution tiers; classify every divergence by the defect classes.

Resolution tiers. Each captured request receives exactly one tier, with precedence **T1** > **T2** > **T3** > **MISS**; the grader records which rule fired.

Miss classification. Every MISS is assigned exactly one class, and this is the measurement that decides whether the approach is viable at all. **MODELABLE**—the gap closes by writing more model (an unmodelled endpoint, field, or error path), so the cost is engineering effort. **PRODUCTION-STATE-DEPENDENT**—the miss requires state that exists only in a real account:

Tier	Definition
T1	*Exact.* Same status code and same decision-relevant fields as the real API, and the agent's control flow is identical.
T2	*Behaviourally equivalent.* Status class and decision-relevant semantics match; only incidental fields differ (identifier format, timestamps, unmodelled fields). Control flow unchanged.
T3	*Divergent but exercising.* The response differs from the real API's in a way that still drives the agent down its write path—typically the model is stricter. Recorded as a fidelity gap.
MISS	The model cannot answer (unmodelled endpoint, parameter, or state), or answers such that the scenario cannot be evaluated.

Table 3. Resolution tiers. T1 and T2 require a documented expectation of the real API's behaviour; absent such a reference a request is graded T3 or MISS, never T1.

a specific pre-existing entity, a live dispute, a provider-side asynchronous event, an inbound webhook. No amount of modelling closes it without real credentials.

Defect classes. Fixed in advance and not extendable after results are seen: (1) *duplicate write*—the same logical mutation applied twice; (2) *unchecked error*—a 4xx treated as success; (3) *stale read*—a decision taken on state cached from before a write; (4) *over-refund*—a refund exceeding the unrefunded amount. A genuine defect fitting none of these is recorded as *unclassified*, reported separately, and does not count toward the primary outcome.

Outcomes. *Primary:* proportion of sampled agents exhibiting at least one class-1–4 defect. *Secondary:* distribution of resolution tiers across all captured requests; MISS rate; the MODELABLE / PRODUCTION-STATE-DEPENDENT split; defects per agent by class. Analysis is descriptive proportions with the denominator stated, and no subgroup analysis beyond these breakdowns.

Falsification. H1 is not supported if fewer than **20%** of sampled agents exhibit any class-1–4 defect. That is reported as a negative result, not reframed.

Kill criterion. If more than **50%** of MISSes are PRODUCTION-STATE-DEPENDENT, the stateful-mock approach cannot be made general by further engineering, and the project pivots rather than continuing to build API models. Both thresholds are fixed now, before any data exists, so that they cannot be moved later.

5 Discussion

The measurement and the artefact are related but independent. Even if AgentCage were never adopted, 20.3% of identifiable write-capable agent tools going unreferenced by tests is a fact about how this software is currently built. And even a perfect coverage figure would not establish that fixture-based tests *detect* write-path defects, which is why Part B is posed as a falsifiable claim rather than a demonstration.

The honest summary of Part A is narrower than its headline: among tools we can mechanically identify in a convenience sample of well-known Python agent projects, about one in five is never named by a test. Whether that generalises to the long tail, to non-Python ecosystems, or to bespoke

agent architectures is unresolved, and the screening rate of 32/45 is the clearest signal of how much remains unmeasured.

6 Availability

Code, the sampling frame, complete per-repository results, and the detector revision history are at `https://github.com/RithikSatarla/agentcage`. The study reproduces with `python -m part_a.run`, requires no credentials, and is re-run weekly in continuous integration. Every statistic in this paper is generated from `part_a/results.json` rather than transcribed.

A Pre-registration checklist

Applies to the Part B evaluation of Section 4.4. Part A is exploratory and makes no pre-registration claim.

Item	Committed
Hypothesis stated before data collection	Yes
Sampling frame and exclusions fixed in advance	Yes
Primary outcome specified, and single	Yes
Secondary outcomes enumerated	Yes
Grading rubric with explicit precedence rules	Yes (T1 > T2 > T3 > MISS)
Defect classes closed to extension	Yes (four classes)
Falsification threshold numeric	Yes (20%)
Kill criterion numeric	Yes (50% of MISSes)
Analysis plan fixed	Yes
Raw evidence published	Yes (all traces committed)
Deviations to be logged with date and reason	Yes
Results known at time of writing	**No — none exist**

Table 4. Pre-registration checklist for Part B.

B Example captured trace

Abridged from the committed trace file, produced by running

```
python -m part_b.demo_double_refund
```

Four requests were captured, three of them writes. Credential headers are replaced at capture time, which is what makes traces committable.

```
{
  "count": 4,
  "traces": [
    { "method": "POST", "path": "/v1/charges",
      "headers": { "authorization": "<redacted>",
                   "content-type": "application/json" },
      "body": "{\"amount\": 5000, \"currency\": \"usd\"}",
```

```
      "status_code": 200 },

    { "method": "POST", "path": "/v1/refunds",
      "body": "{\"charge\": \"ch_00000001\", \"amount\": 5000}",
      "status_code": 200 },

    { "method": "POST", "path": "/v1/refunds",
      "body": "{\"charge\": \"ch_00000001\", \"amount\": 5000}",
      "status_code": 400,
      "response_body": "{\"error\": {\"type\": \"invalid_request_error\",
                       \"code\": \"charge_already_refunded\"}}" },

    { "method": "GET", "path": "/v1/charges/ch_00000001",
      "status_code": 200,
      "response_body": "{... \"amount_refunded\": 5000 ...}" }
  ]
}
```

The two refund requests are byte-identical in method, path, and body. Only a mock that holds state can answer them differently, and answering them differently is what correctness requires.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629*, 2022.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv:2302.04761*, 2023.

[3] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. *arXiv:2305.15334*, 2023.

[4] Y. Qin et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv:2307.16789*, 2023.

[5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv:2310.06770*, 2023.

[6] S. Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv:2307.13854*, 2023.

[7] X. Liu et al. AgentBench: Evaluating LLMs as Agents. *arXiv:2308.03688*, 2023.

[8] G. Mialon et al. GAIA: A Benchmark for General AI Assistants. *arXiv:2311.12983*, 2023.

[9] Y. Ruan et al. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *arXiv:2309.15817*, 2023.

[10] E. Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *arXiv:2406.13352*, 2024.